

# Keeping Midas in the Sandbox: Using AI Coding Agents with Sensitive Research Data

David Wegmann

Ahrabhi Kathirgamalingam

Yuru Li

Paul Pressmann

## Learning Objectives

By the end of this tutorial, you will be able to:

1. Explain why AI coding agents create privacy, containment, validity, reproducibility, and disclosure risks in sensitive-data research.
2. Describe the Midas workflow: a Docker Sandbox-visible Midas directory, a GitHub code bridge, and an agent-free vault directory where real data are stored and analyzed.
3. Set up project rules that keep protected data away from the coding agent.
4. Use GitHub as a code-only bridge that supports review, version history, and reproducibility without moving real data into the agent-visible environment.
5. Create and manually review a sanitized data-shape description that helps the coding agent write scripts without seeing real observations.
6. Use mock data in the Midas directory to develop and test analysis code before running that code on real data in the vault.
7. Review AI-generated code for validity, methodological appropriateness, and possible data-exfiltration risks before running it in the protected environment.
8. Document AI-agent use clearly for collaborators, reviewers, and readers, including what the agent could access, what it could not access, and who remained responsible for the final analysis.

## Target Audience

This tutorial is for computational social science and communication researchers who want help from AI coding agents while working on projects that involve sensitive, restricted, copyrighted, participant-derived, or otherwise protected data.

It assumes basic familiarity with Git, GitHub, command-line work, and script-based analysis in Python, R, or a similar language. It does not assume prior experience with Docker Sandboxes or with AI coding agents.

The tutorial is also written for reviewers, collaborators, supervisors, and research support staff who need to understand what it means when a manuscript says that an AI coding agent helped write code but did not generate the results or access sensitive data.

It follows the Methods Hub tutorial format by making the audience, prerequisites, workflow steps, limitations, and references explicit (GESIS Leibniz Institute for the Social Sciences 2026b, 2026a).

## Prerequisites

You need:

- a **GitHub** account
- a **Docker log-in** (can be authenticated with your GitHub account)
- a **computer A** with internet access, running:
  - your programming language of choice, i.e. Python or R
  - Git or an equivalent Git host approved for your project.
  - Docker Sandboxes
- Access to a Docker Sandboxes-supported **AI-agent**.
- A second **computer B** approved to store your real data, e.g. a server, with internet access, running:
  - your programming language of choice, i.e. Python or R
  - Git

## Duration

Around three hours for reading and adapting the workflow to a new project. More time is needed if the protected vault environment requires institutional setup or review.

## Social Science Use Case

Today social and communication research often analyzes platform posts, donated digital trace data, interview transcripts, or restricted archive records. Such data are valuable because they capture social behavior at high resolution, but they are also methodologically and ethically difficult because they were often not originally produced for research and may contain contextual details that identify people directly or indirectly (Ohme et al. 2024). Usernames,

free text, timestamps, links, images, locations, demographic information, behavioral traces, and combinations of otherwise ordinary variables can all become sensitive in context. Recent work on privacy-preserving and privacy-by-design data workflows therefore emphasizes that safeguards should be built into the research pipeline itself rather than added only after data have already circulated (Oh et al. 2025).

AI coding agents can be extremely useful in this setting, especially because they can accelerate ordinary programming, analysis, and documentation work (“Agentic Coding,” n.d.; Engzell and Wilmers 2026). The problem is that they do not only “help with code” in the abstract. They inspect files, read terminal output, process error messages, and reason from whatever appears in the workspace. For most current agents, this means that the inspected material is transferred to remote provider infrastructure, where it may be subject to external processing, logging, retention, or other governance arrangements. A single pasted row, stack trace, notebook output, screenshot, or log file can therefore expose protected material. These risks align with broader concerns about privacy leakage in large language model systems, where sensitive information may enter prompts, files, logs, or outputs (Chen et al. 2025).

This makes the use of coding agents in social science research risky for projects involving sensitive, restricted, copyrighted, or participant-derived data. The issue becomes sharper in agentic systems that can inspect workspaces, call tools, execute commands, and act across connected resources. For such systems, classical security principles such as least privilege, defense-in-depth, and clear access boundaries are directly relevant (He et al. 2024; Zhang et al. 2025). Coding assistance must therefore be organized around containment from the start: the agent may help with code, documentation, tests, and mock data, but it must be kept away from protected data and protected outputs. This also supports clearer disclosure, because researchers can state not only that AI assistance was used, but also what the agent was allowed to see, what it was kept away from, and how the resulting code was reviewed (Kostygina et al. 2023).

## Core Principle

The problem is like the myth of King Midas. Midas’ power and problem was that everything he touched turned to gold: an ability of great utility in monetary regards, but much more problematic when it came to food, drink, or his family. His gift was valuable only as long as he touched the right things.

The touch of coding agents is similarly powerful. They can write code, confidently use obscure APIs, and generate elaborate data-processing pipelines far faster than human researchers could do manually. Used well, they allow researchers to test ideas, prototype analyses, document code, and improve workflows at a speed that would otherwise be difficult to achieve. In social science, where data are often difficult and costly to collect and can lose relevance quickly, coding agents promise to help researchers get more out of the data they can responsibly use.

But their touch is also the source of the problem. Currently, almost all coding agents operate through remote provider infrastructure. Material they inspect is therefore transmitted beyond the local workspace and becomes subject to external processing, logging, retention, or governance arrangements. Once protected research material enters that channel, researchers may no longer be able to control where it goes, how it is processed, or what traces of it remains.

This tutorial refers to the directory that the coding agent can touch and act in as the *Midas directory*. The *vault directory* is the separate protected location where real data live. The goal of the workflow is not to make the coding agent harmless, but to mitigate its risks by separating it from the real data.

The core rule is therefore:

Let the coding agent help with code, but do not let it touch protected data.

In practical terms, the data are stored in a vault directory. The coding agent is kept separate from that vault directory and is confined to a sandboxed Midas directory. The researcher communicates the data's shape to the coding agent without revealing the data itself. The coding agent develops analysis scripts in the Midas directory. The researcher transfers the reviewed scripts from the Midas directory to the vault directory and runs them on the real data.

This architecture strongly reduces privacy and containment risks. It features two protection layers: a Docker Sandbox that confines the AI agent's access to the directory it is working in and a fully separate computer that stores the sensitive data and is never accessed by the coding agent (Docker Inc. 2026). This two-layered approach protects against setup mistakes and reduces the risk of human error by making the separation between the AI-agent environment and the sensitive-data environment explicit.

The architecture does not, by itself, solve validity, interpretation, or accountability risks. Those still require human review, transparent documentation, and independent validation. On the upside, however, this approach creates a code repository that can be made accessible to reviewers and readers, supporting transparency and reproducibility.

## The Three-Part Workflow

The workflow separates the coding agent, the code bridge, and the real data.

| Midas directory<br>-----                                                                    |    | GitHub bridge<br>-----                                        |    | vault directory<br>-----                                                                  |
|---------------------------------------------------------------------------------------------|----|---------------------------------------------------------------|----|-------------------------------------------------------------------------------------------|
| Docker Sandbox visible<br>AI coding agent allowed<br>code and documentation<br>no real data | -> | reviewed code only<br>version history<br>pull requests/review | -> | no coding agent<br>real data live here<br>protected outputs stay here<br>final validation |

## Midas Directory

The Midas directory is the project folder made visible inside a [Docker Sandbox](#), a tool specifically designed to restrict coding agents' access on the device they are employed on. The coding agent has access to the sandbox and only the sandbox. The contents of the sandbox are considered public for all intents and purposes. This can include code, documentation, agent rules, notes that are safe to share, and safe mock data.

It must not contain real data or traces of it such as real-data outputs, credentials, protected logs, screenshots, notebooks with real observations, or unsanitized error reports.

## GitHub Bridge

GitHub is the bridge between the Midas directory and the vault directory. Once the agent has created the analysis code, the code is pushed to GitHub. On GitHub the code can be accessed by the researcher and pulled into the vault directory to run the analysis. Optionally, it can also be made available to reviewers and readers of the research project output later in the project to facilitate transparency and reproducibility. As with the Midas directory, the GitHub repository is considered public and cannot contain sensitive data.

## Vault Directory

The vault directory is the protected location where real data are stored and analyzed. It may be an institutional server, secure VM, restricted research workspace, or any other environment approved for the data and capable of running the scripts developed by the agent.

The vault contains real data and protected outputs. It pulls code from GitHub, runs the analysis, and stores outputs. At no point is it accessed by any coding agent.

## Step 1: Create Your Project Repository from the Template

Since this workflow is not the regular way of using coding agents, it is useful to start from a minimal template that already contains the most important boundaries:

- `AGENT_RULES.md`, which tells the agent what it may and may not access;
- `PROJECT_BRIEF.md`, which gives the agent safe project context;
- `.gitignore`, which blocks common data, output, log, credential, and generated report paths;
- `scripts/`, where the agent can create analysis code;
- `tools/`, which contains the vault-side data-shape report scripts.

The easiest route is to open the [2U2N/midas\\_template](#) repository on GitHub and click **Use this template**. Create a new repository under your own GitHub account. Do not work directly in the template repository. The new repository is your project repository; it is the code-only bridge between the Midas directory and the vault directory.

First set up the repository in the vault on *computer B*. For the first setup, use `git clone`:

```
git clone https://github.com/YOUR-USER/YOUR-PROJECT.git
cd YOUR-PROJECT
```

## Step 2: Describe the Data Without Exposing It

The agent needs to understand the shape of the data before it can write useful scripts. But the agent must not see the data itself. In this step we create a detailed `PROJECT_BRIEF.md` for the agent to read that communicates the necessary details without exposing any sensitive data. For that we

- create a detailed but sanitized structural description of the data in the vault
- review the data report manually to prevent data leakage
- manually note the goals and scope of the research project
- push the reviewed `PROJECT_BRIEF.md` through GitHub.

### Create a Detailed but Sanitized Description of the Data in the Vault

With sufficient understanding of the shape and features of a dataset, a coding agent can write an analysis script for that data without accessing it. In this step we lay the foundation for that understanding by gathering as much information on the data that is supposed to be analyzed without exposing its sensitive contents.

This can be done manually by writing up file names, row and column counts, data types, and similar structural details. But that can be tedious. Therefore, the template contains scripts in the folder `tools/` that, in a data-safe manner and without the use of any coding agent, scan a file or folder and create a report that is detailed but privacy preserving.

Create the data report by running either the Python or R script in `tools/`, pointing it at the real data location. For Python:

```
python3 tools/make_data_shape_report.py \
  --input /path/to/your/data \
  --output data_shape_report.md
```

For R:

```
Rscript tools/make_data_shape_report.R \  
  --input /path/to/your/data \  
  --output data_shape_report.md
```

The scripts work out of the box for a host of common data formats. Support for some formats depends on optional libraries. If these libraries are not installed, the script records that the file could not be parsed. In those cases users need to install the libraries for the specific file formats they want to inspect and run the script again.

## Review the Data Report

Both data report scripts create a Markdown report, `data_shape_report.md`, with approximate row counts, column names, inferred column types, missingness buckets, and safety flags. They are designed to avoid printing real data. Nevertheless their output should be critically reviewed.

Open `data_shape_report.md` and copy all contents that are free of sensitive data to the section *Sanitized Data Shape Report* of `PROJECT_BRIEF.md`. To prevent data leakage, `data_shape_report.md` is included in the template `.gitignore` and will by default not be transferred to the Midas directory.

## Manually Note the Goals and Scope of the Research Project

`PROJECT_BRIEF.md` contains additional sections:

- Research Goal
- Programming Language
- Intended Analysis

Fill these sections with the details of your research project. This document will be the main source of context for the coding agent to develop the analysis code. Generally, thoroughness at this stage of the project leads to better results. However, be mindful that the coding agent will treat any information provided here as project context throughout the remainder of the work.

Coding agents tend to get confused and produce worse results when their context, such as the `PROJECT_BRIEF.md`, contradicts other instructions. Therefore, it can be advisable to leave out some details here and communicate these instructions in chat prompts during direct interaction with the coding agent later in the project. This could for example include the exact statistical tests to be conducted.

## Push the Data Description through GitHub

Save `PROJECT_BRIEF.md`. Then commit and push the reviewed `PROJECT_BRIEF.md`:

```
git status
git add PROJECT_BRIEF.md
git commit -m "Add sanitized data shape description"
git push
```

## Step 3: Pull the Project to Computer A and Start Midas

On *computer A*, clone your project repository:

```
git clone https://github.com/YOUR-USER/YOUR-PROJECT.git
cd YOUR-PROJECT
```

This folder will be the Midas directory. Treat everything in it as public and visible to the AI-agent provider. Do not add any sensitive data or traces of sensitive data like credentials, screenshots, notebooks with real outputs, raw logs, or unsanitized errors.

Log into your Docker Sandbox account, as described in [Docker's own manuals](#), by typing this in the CLI.

```
sbx login
```

`sbx login` opens a browser for Docker OAuth. On first login (and after `sbx policy reset`), the CLI prompts you to choose a default network policy for your sandboxes:

Choose a default network policy:

1. Open - All network traffic allowed, no restrictions.
2. Balanced - Default deny, with common dev sites allowed.
3. Locked Down - All network traffic blocked unless you allow it.

Use `↑/↓` to navigate, `Enter` to select, or press 1-3.

Start the AI agent of your choice. Here is a [list of the AI-agents Docker Sandbox supports](#) (Docker Inc. 2026). Clicking on them reveals details about the respective authentication procedure.



```
sbx run [agent] .
```

Docker Sandboxes run agents inside isolated sandbox environments. Docker documents [isolation across the sandbox VM, network, Docker Engine, workspace, and credentials](#) (Docker Inc. 2026). A default sandbox blocks host filesystem access outside the workspace, host Docker daemon access, host localhost, communication between sandboxes, raw TCP, UDP, ICMP, and traffic to private or link-local IP ranges; see Docker’s [default security posture](#).

**Note:** This does not make it safe to put real data in the Midas directory. It just means that the agent can’t escape it and touch data on your device that is stored outside of its sandbox.

## Step 4: Instruct the Coding Agent

At this point you can interact with the coding agent as usual and instruct it to develop your analysis pipeline. It is advisable start with these prompts in order:

- “Familiarize yourself with this directory”
  - Given the `AGENT_RULES.md`, `PROJECT_BRIEF.md`, and `README.md`, this should provide the agent with sufficient context to understand your research project and its role in it.
- “I want to work on developing the analysis code for the project outlined in `PROJECT_BRIEF.md`. Create mock datasets for all datasets described in the Sanitized Data Shape Report section of `PROJECT_BRIEF.md`.”
  - Monitor that the mock-data does indeed resemble your real data, for example by tasking a second instance of your coding agent to validate congruence between the mock data and the description in `PROJECT_BRIEF.md`.

At this point you can continue working with the coding agent to co-develop the data processing pipeline that suits your project, from data cleaning to analysis output and data visualization. The agent will use the mock data to produce mock results. Continue to instruct the agent and adapt the code manually until the mock output has the expected structure.

## Step 5: Review and Push Midas-Created Code

Review the code to ensure that it not only produces good-looking results, but also goes through the data processing steps required to produce valid results. You may employ the coding agent itself to document and explain the code. However, you will have to take on the responsibility of the final validation. It cannot be outsourced to the agent.

**Note:** There is a small but non-zero chance that the agent will develop code that exposes your data beyond your secure environment. It may, for example, employ external APIs in ways that include transferring your data to third parties. This must be prevented and should be easy to spot during code review.

Once satisfied with output and code push the repository to GitHub.

```
git add .  
git commit -m "Add analysis scripts"  
git push
```

## Step 6: Run Analysis in the Vault

In the vault environment on *computer B*, pull the reviewed code from GitHub:

```
cd YOUR-PROJECT  
git pull
```

Then execute the analysis scripts. Depending on the code you will likely need to provide the paths to your datasets and output directories.

Code execution on real data may fail in ways that the coding agent in the Midas directory did not anticipate. In those cases communicate the issue via chat-prompt to your coding agent. Only paste real error-logs if necessary and after strict review of their contents, they often include snippets of real data.

Once you or your coding agent have addressed the issue in the Midas directory, again push the repository to GitHub and from there pull it into the vault directory to run it again.

## Step 7: Transparency, Reproducibility and Disclosure

At this point, the project repository contains the analysis code, documentation, mock data, project rules, and sanitized data description, but not the protected data themselves. This makes it a strong basis for a transparency and reproducibility repository (Kostygina et al. 2023).

Before publication or review, the repository can be cleaned up and documented for external readers. Depending on the project, this may include improving the README, documenting the expected vault-side inputs and outputs, keeping or expanding mock data, adding package requirements, and explaining which parts of the analysis can be reproduced publicly and which require authorized access to protected data.

Reproducibility does not require making protected data public. In this workflow, readers and reviewers can inspect the code, understand the data structure it expects, run the pipeline on mock data, and evaluate the containment architecture. Authorized researchers with access to the protected vault data can reproduce the full analysis in the vault environment (Kostygina et al. 2023; Engzell and Wilmers 2026).

Agent use should also be disclosed clearly. A disclosure does not need to overstate the role of the coding agent, but it should say what the agent helped with, what it could not access, and who remained responsible for the analysis (Kostygina et al. 2023). For example:

Analysis code was developed with AI-assisted coding under a data-containment workflow. The agent (product, model version, date of usage) had access only to mock data and sanitized structural documentation, not to raw data or protected outputs. Final analyses were run in an agent-free environment. Further details on the workflow are provided in [this tutorial / repository / appendix].

## Limitations

This workflow reduces the risk that AI coding agents access protected research data, but it does not make such projects risk-free. Docker Sandboxes help keep the coding agent away from the host system and from files outside the chosen workspace, but the agent can still see everything inside the Midas directory. GitHub creates version history and review points, but it is not protected storage. The vault protects real data only if researchers keep agents, logs, screenshots, notebooks, and outputs inside the approved environment (Chen et al. 2025; He et al. 2024; Zhang et al. 2025).

The workflow also does not remove the need for ethical, legal, and institutional review. Some projects may involve data that require stricter controls than the architecture described here, and researchers remain responsible for checking whether this setup is acceptable under their data agreements, ethics approvals, institutional policies, and applicable law.

Finally, containment is not the same as validity. The coding agent may produce code that is inefficient, incorrect, misleading, or inappropriate for the research question. Human researchers must therefore review, test, and validate the code before it is run on real data and remain responsible for the final analysis, results, and interpretation.

## References

“Agentic Coding.” n.d. Missing Semester. Accessed June 24, 2026. <https://missing.csail.mit.edu/2026/agentic-coding/>.

- Chen, Kang, Xiuze Zhou, Yuanguo Lin, Shibo Feng, Li Shen, and Pengcheng Wu. 2025. “A Survey on Privacy Risks and Protection in Large Language Models.” *Journal of King Saud University Computer and Information Sciences* 37 (7): 163. <https://doi.org/10.1007/s44443-025-00177-1>.
- Docker Inc. 2026. “Docker Sandboxes.” Docker Documentation. <https://docs.docker.com/ai/sandboxes/>.
- Engzell, Per, and Nathan Wilmers. 2026. “The Paper Factory.” 24xfq\_v1. Pre-published May 5. [https://osf.io/preprints/socarxiv/24xfq\\_v1/](https://osf.io/preprints/socarxiv/24xfq_v1/).
- GESIS Leibniz Institute for the Social Sciences. 2026a. “Method Hub’s Guidelines for Tutorials.” GESIS-Methods-Hub. <https://github.com/GESIS-Methods-Hub/guidelines-for-tutorials>.
- GESIS Leibniz Institute for the Social Sciences. 2026b. “Methods Hub.” <https://methodshub.gesis.org/>.
- He, Yifeng, Ethan Wang, Yuyang Rong, Zifei Cheng, and Hao Chen. 2024. “Security of AI Agents.” Version 2. Pre-published June 20. <https://doi.org/10.48550/arXiv.2406.08689>.
- Kostygina, Ganna, Yoonsang Kim, Zachary Seeskin, Felicia LeClere, and Sherry Emery. 2023. “Disclosure Standards for Social Media and Generative Artificial Intelligence Research: Toward Transparency and Replicability.” *Social Media + Society* 9 (4): 20563051231216947. <https://doi.org/10.1177/20563051231216947>.
- Oh, Nick, Giorgos D. Vrakas, Siân J. M. Brooke, Sasha Morinière, and Toju Duke. 2025. “PETLP: A Privacy-by-Design Pipeline for Social Media Data in AI Research.” *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society* 8 (2): 1926–38. <https://doi.org/10.1609/aies.v8i2.36685>.
- Ohme, Jakob, Theo Araujo, Laura Boeschoten, et al. 2024. “Digital Trace Data Collection for Social Media Effects Research: APIs, Data Donation, and (Screen) Tracking.” *Communication Methods and Measures* 18 (2): 124–41. <https://doi.org/10.1080/19312458.2023.2181319>.
- Zhang, Kaiyuan, Zian Su, Pin-Yu Chen, Elisa Bertino, Xiangyu Zhang, and Ninghui Li. 2025. “LLM Agents Should Employ Security Principles.” Pre-published May 29. <https://doi.org/10.48550/arXiv.2505.24019>.